Progetto DAPT W12D4

Giovanni M. Martino

October 26, 2026

DAPT0326IT

Scenario e obiettivo

L'azienda ToysGroup distribuisce giocattoli tramite una rete di rivenditori in diverse aree geografiche del mondo. I prodotti sono classificati in categorie, i mercati in regione di vendita. Ogni vendita collega un prodotto a una regione.

ToysGroup vende prodotti di tre categorie (*Outdoor*, *Giochi da tavolo*, *Giocattoli*) tramite una rete di 300 rivenditori in 9 paesi europei, raggruppati in tre aree di vendita (Sud, Centro e Nord Europa).

Il progetto che stiamo per presentare analizza le transazioni avvenute con tre obiettivi principali: capire l'andamento del fatturato, prevedere i rivenditori a rischio di abbandono e raggruppare i rivenditori in segmenti omogenei.

I Dati

ToysGroup ci ha fornito un dataset di partenza formato da quattro file che descrivono il proprio scenario:

- Due file con le anagrafiche, ovvero, i dati relativi ai prodotti, alle categorie di appartenenza e il listino prezzi; i dati relativi alle regioni di vendita, così da collegare la vendita a un'area geografica specifica.
- Un file relativo alle vendite, con tutte le transazioni effettuate da ToysGroup; collegando quindi una vendita alla relativa anagrafica.
- Un file con i rivenditori, quindi, i dati relativi agli ordini, agli sconti, ai reclami ricevuti da ToyGroup.

Sul metodo di lavoro

Lo strumento di lavoro che abbiamo scelto per svolgere i task che ci sono stato assegnati è *Python*, ovvero, un linguaggio di programmazione di alto livello, interpretato e open source, molto apprezzato per la sua sintassi molto chiara, lineare e vicina al linguaggio naturale.

È stato ideato dal programmatore olandese Guido van Rossum alla fine degli anni '80, per poi essere rilasciato ufficialmente nel 1991. Ai i più attenti, tuttavia, il nome non può lasciare indifferenti: deriva, infatti, dalla famosa serie comica della BBC "Monty Python's Flying Circus" o, semplicemente, *Monty Python*, con Graham Chapman, John Cleese, Eric Idle, Terry Jones, Michael Palin e Terry Gilliam.

In ciò che segue, quindi, lavoreremo con Python.

Su Python

Al fine di favorire la comprensione del progetto che stiamo presentando, dei risultati ottenuti e, soprattuto, del quadro complessivo del lavoro, abbiamo scelto di non soffermarci in modo dettagliato sulla spiegazione del codice che abbiamo utilizzato in Python.

Lavorando con un linguaggio di programmazione come Python, per quanto di alto livello, è necessario scegliere un metodo di presentazione consono che renda chiaro e comprensibile quanto abbiamo svolto. Abbiamo scelto, allora, di non soffermarci sul *come* abbiamo lavorato ma bensì di spiegare il *cosa* utilizziamo e per quale motivo.

Sulla presentazione

Abbiamo scelto, così, di introdurre i concetti che ci abbiamo impiegato in modo progressivo: ci è sembrato il metodo più consono per una presentazione diretta, come in questo caso, a un pubblico non specialista.

Da qui in avanti, dunque, le possibile oscurità sono da imputare esclusivamente all'autore della presentazione: sono gradite, quindi, eventuali obiezioni o chiarificazioni su quanto abbiamo fatto.

E ora, però, di iniziare.

Task 1a, *Python base sui dati forniti*

Il primo task che ci è stato chiesto di affrontare è diviso in quattro parti e riguarda, in modo preliminare, le prime analisi sulla struttura dei dati che ci sono stati forniti da ToysGroup. Il metodo d'indagine, almeno in questa prima fase, prevede di utilizzare Python con le sue funzioni base.

- Lettura dei primi file che sono stati forniti, in particolare quelli relativi a *Products* e *Sales*.
- L'assegnazione, per ogni prodotto, a una fascia di prezzo: *Bassa* sotto 20 €, *Media* fino a 60 €, *Alta* oltre.
- Realizzare una funzione che calcoli la somma dei *SalesAmount* della categoria di pertinenza, non calcolando i valori mancanti
- Infine, contare le transazioni necessarie per superare 10.000 € di fatturato cumulato, mostrando le prime cinque transazioni con vendite

Il primo Task

Inizialmente, abbiamo letto i primi due file, ovvero, *Products* e *Sales* per poi leggere anche, *Regions*. Così facendo, in Python, abbiamo visualizzato i primi dati dove abbiamo visto che le uniche tre categorie di giochi sono: *Giochi da tavolo*, *Outdoor* e *Giocattoli*.

Abbiamo poi assegnato, ad ogni prodotto, una fascia di prezzo.
Nell'esempio sotto, abbiamo preso i primi cinque prodotti risultanti.

- Bici Junior, Outdoor, 129.0, Fascia Alta
- Bici BMX, Outdoor, 189.0, Fascia Alta
- Monopattino, Outdoor, 79.0, Fascia Alta
- Pallone Calcio, Outdoor, 19.9, Fascia Bassa
- Racchette Beach, Outdoor, 14.5, Fascia Bassa

Il primo Task, seconda parte

Abbiamo poi realizzato una *funzione fatturato categoria* che ha come argomenti le vendite, i prodotti e le categoria) sommando poi gli importi di vendita, ovvero i *SalesAmount* della categoria, saltando i valori mancanti, restituendo solo il totale.

Abbiamo, infine, calcolato le transazioni necessarie, ovvero 56, a superare 10.000 € di fatturato cumulato; abbiamo poi calcolato la somma totale del fatturato, cioè, 10.164.8 €.

Infine, abbiamo visto un esempio di anagrafica delle transazioni:

- SalesID: 1194, ProductID: 6, RegionID: 7, ResellerID: 283, SalesDate: 2023-01-01, Quantity: 4.0, SalesAmount: 71.6

Task 2, *Lettura e ispezione dei dati*

Il secondo Task ci permette di entrare più nel vivo dell'analisi dello scenario ToysGroup attraverso una prima ispezione dei dati, così da descriverne contenuto e struttura.

- Lettura completa dei quattro file del dataset impiegando i quattro metodi fondamentali per descrivere la struttura dei dati.
- Calcolare le prime funzioni statistiche sul dataset, minimo, massimo e media calcolando, poi, il prezzo unitario di ogni vendita.
- Effettuare un'indagine che ci dia i prodotti di una categoria, a nostra scelta, con *ListPrice* superiore a 30 € mostrando le prime cinque transazioni.

Lettura dei dataset

Abbiamo cominciato la lettura del dataset importando in Python i relativi file che ToysGroup ci aveva dato. Ecco, qui, cosa abbiamo estratto da una prima lettura

- *products* ha 12 righe e 4 colonne, per 12 voci totali con *ProductID*, *ProductName*, *Category*, *ListPrice*.
- *sales* ha 2000 righe e 7 colonne con *SalesID*, *ProductID* , *RegionID*, *ResellerID*, *SalesDate*, *Quantity*, *SalesAmount*.
- *resellers* ha 300 righe e 7 colonne con *ResellerID*, *RegionID*, *Ordini12m*, *GiorniUltimoOrdine* , *ScontoMedio*, *Reclami* e *Churn*.
- *regions*, infine, ha 9 righe e 3 colonne, con *RegionID*, *State* e *SalesRegion*

Tabelle statistiche relative

Abbiamo poi calcolato minimo, massimo e media di *Quantity* e *SalesAmount* e calcolato il prezzo unitario di ogni vendita.

	Quantity	SalesAmount
count	1900.00	1900.00
mean	4.97	294.78
std	2.21	305.26
min	1.00	12.50
25%	3.00	89.00
50%	5.00	177.00
75%	6.00	387.00
max	14.00	2457.00

Table: Statistiche descrittive di Quantity e SalesAmount

Prodotti con prezzo superiore a 30

Abbiamo, infine, creato una tabella per i prodotti di una categoria con *ListPrice* superiore a 30 €; con questa sotto, poi, mostriamo i risultati da 10 a 19 e le prime tre colonne di sales.

SalesID	ProductID	RegionID	Colonna4
10	218	1	8
11	1136	1	9
12	1326	3	4
13	1063	9	7
14	1606	10	8
15	1283	12	1
16	98	3	4
17	156	6	8
18	1642	12	5
19	1644	4	7

Table: Dati delle transazioni e regioni

Task 3a, *Dati mancanti, filtri e ordinamento*

Il successivo Task ci chiede di rendere il DataFrame delle vendite pronto per la nostra 'analisi, trattando i valori mancanti e andando a isolare alcuni dei sottoinsiemi richiesti.

- Enumerare i valori mancanti, chiamai NaN, nelle colonne del DataFrame calcolando, poi, la percentuale totale sul numero totale delle righe.
- Per riempire i valori mancanti, utilizzare prima il criterio della *mediana* e poi eliminare le righe senza *SalesAmount* dichiarandone il numero.
- Estrarre alcune transazioni da *SalesAmount*, quelle superiori al 90° percentile, ordinandole poi per data crescente e importo crescente.

Valori mancanti

La prima parte del terzo task ci ha chiesto di enumerare i valori mancanti calcolandone, poi, una percentuale: abbiamo trovato valori mancanti solo in *SalesAmount* e *Quantity*, entrambi 100, con una percentuale quindi del 5 rispetto al totale:

Colonna	**Valori Assoluti**	**Percentuale (%)**
SalesID	0	0.0
ProductID	0	0.0
RegionID	0	0.0
ResellerID	0	0.0
SalesDate	0	0.0
Quantity	100	5.0
SalesAmount	100	5.0

Table: Analisi dei valori mancanti per colonna

Seconda parte del terzo task

Abbiamo poi operato un taglio dei valori mancanti rimuovendo le 100 righe con valori mancanti e lasciando le altre. Il computo finale è di 1900, così, abbiamo ordinato le transazioni per importo crescente e data crescente come segue.

SalesID	ProductID	RegionID	ResellerID	SalesDate	Quantity
32	1045	2	4	82	2023-01-16
49	1872	2	7	204	2023-01-25
52	1639	1	3	299	2023-01-27
102	296	2	9	202	2023-02-25
118	1462	2	2	224	2023-03-03

Table: Estratto delle prime 5 transazioni

Task 3b, *Aggregazioni e combinazione dei DataFrame*

La seconda parte del terzo Task ci chiede di costruire ostruire una tabella unica delle vendite e calcolare il fatturato per le principali chiavi di analisi.

- Unire le tabelle *Sales* con *Product* e *Regions* verificando poi che il numero di righe risultante nella nuova tabella si uguale, ovviamente, a quella di *Sales*.
- Calcolare, poi, il fatturato totale, quantità totale e numero di transazioni per *Category* e per *SalesRegion* in modo decrescente.
- Operare una separazione interna alle vendite del 2025 tramite un filtro, unendo poi, nuovamente il tutto.

I risultati del Task 3b

Per prima cosa, una volta operato il taglio dei valori mancanti, abbiamo effettivamente verificato che con 100 righe in meno, abbiamo 1900 valori all'interno di *Sales*. Per verificare ulteriormente, in Python, abbiamo operato prima una divisione tra importi del 2025 e gli altri importi, per poi unirli nuovamente: lo scopo era la conferma delle 1900 transazioni complessive.

Così, nel 2025 abbiamo avuto 640 transazioni, le restanti erano 1260 con somma complessiva sempre di 1900 che conferma il risultato nella nostra analisi.

Successivamente, abbiamo calcolato, nelle slide che seguono, due tipologie di fatturato.

Fatturato per categoria

SalesRegion	Fatturato	Quantità	Transazioni
Centro Europa	201062.2	3413.0	698
Nord Europa	190189.0	3175.0	633
Sud Europa	168824.3	2853.0	569

Table: Fatturato, quantità e transazioni per regione di vendita

Fatturato per Regione di vendita

SalesRegion	Fatturato	Quantità	Transazioni
Centro Europa	201062.2	3413.0	698
Nord Europa	190189.0	3175.0	633
Sud Europa	168824.3	2853.0	569

Table: Fatturato, quantità e transazioni per regione di vendita

Task 3c. *Serie temporali di vendita*

L'ultima parte del terzo Task riguarda l'analisi dell'andamento delle vendite nel tempo a partire dal campo *SalesDate*

- Operare la conversione dei dati in *SalesDate* in Python sul tipo di dato *datetime*.
- Lavorare sul nuovo campo con le informazioni relative ad anno, mese e giorno della settimana.
- Calcolare il fatturato mensile su anno e mese e la variazione percentuale tra mesi consecutivi.
- Stabilire un criterio di vendita per cui se la vendita è avvenuta entro 180 giorni dall'ultima data presente nel dataset abbiamo una risposta positiva, negativa altrimenti.

Le richieste del Task 3c

I risultati del nostro calcolo dalle richieste precedenti le riassumiamo come segue:

- SalesDate convertita in datetime64, ovvero, abbiamo cambiato tipo di dato nel nostro linguaggio di programmazione.
- La Serie mensile di vendite è di 36 righe (da gennaio 2023 a dicembre 2025).
- Il mese con il fatturato massimo è maggio 2025, con 29.368,30 €.
- Ultima data del dataset: 25-12-30.

Le nuove colonne del dataset

SalesID	ProductID	RegionID	ResellerID	SalesDate	Quantity
1194	6	7	283	2023-01-01	4.0
1747	12	3	281	2023-01-02	2.0
947	7	8	166	2023-01-04	6.0
577	3	4	285	2023-01-04	3.0
78	3	8	107	2023-01-04	3.0

Table: Dettaglio transazioni - Dati di vendita (Parte 1)

SalesID	Anno	Mese	Giorno	data_recente
1194	2023	1	6	False
1747	2023	1	0	False
947	2023	1	2	False
577	2023	1	2	False
78	2023	1	2	False

Table: Dettaglio transazioni - Componenti temporali (Parte 2)

Task 4. *Visualizzazione e storytelling*

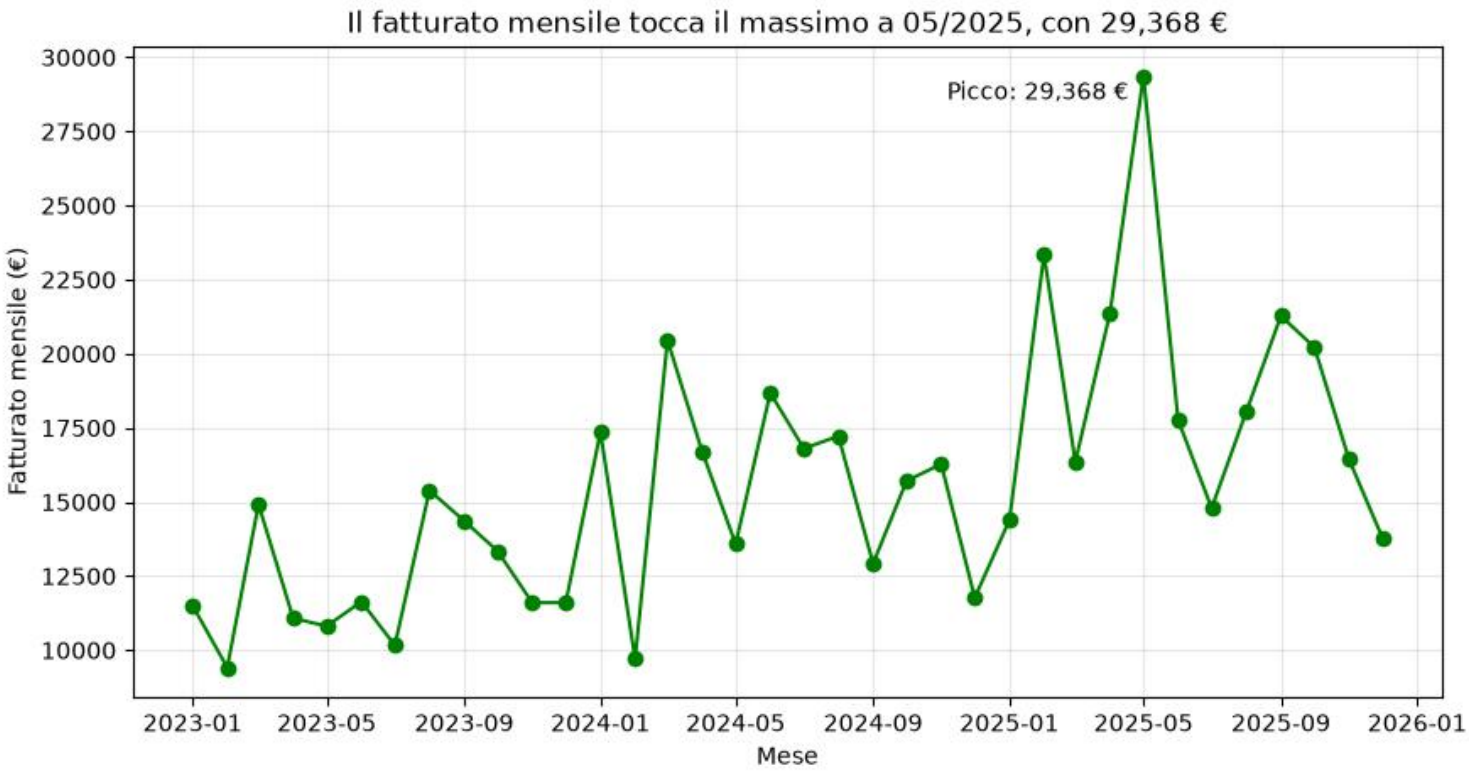
La seconda parte del nostro lavoro con ToysGroup comincia con la rappresentazione grafica dell'analisi in corso attraverso le domande di business a cui, il nostro lavoro, risponde.

- Realizzare un grafico a linee del fatturato mensile delle serie temporali di vendita (Task 3c), con titolo, etichette degli assi con unità (€) e mese di picco evidenziato con un'annotazione.
- Generare un *boxplot* di *SalesAmount* per categoria e la *heatmap* della matrice di correlazione delle variabili numeriche di *resellers*.
- Creare un grafico a barre interattivo del fatturato per stato, con quantità totale e numero di transazioni.

Fatturato mensile più alto

Come cambia il fatturato mensile? Quando raggiunge il massimo?

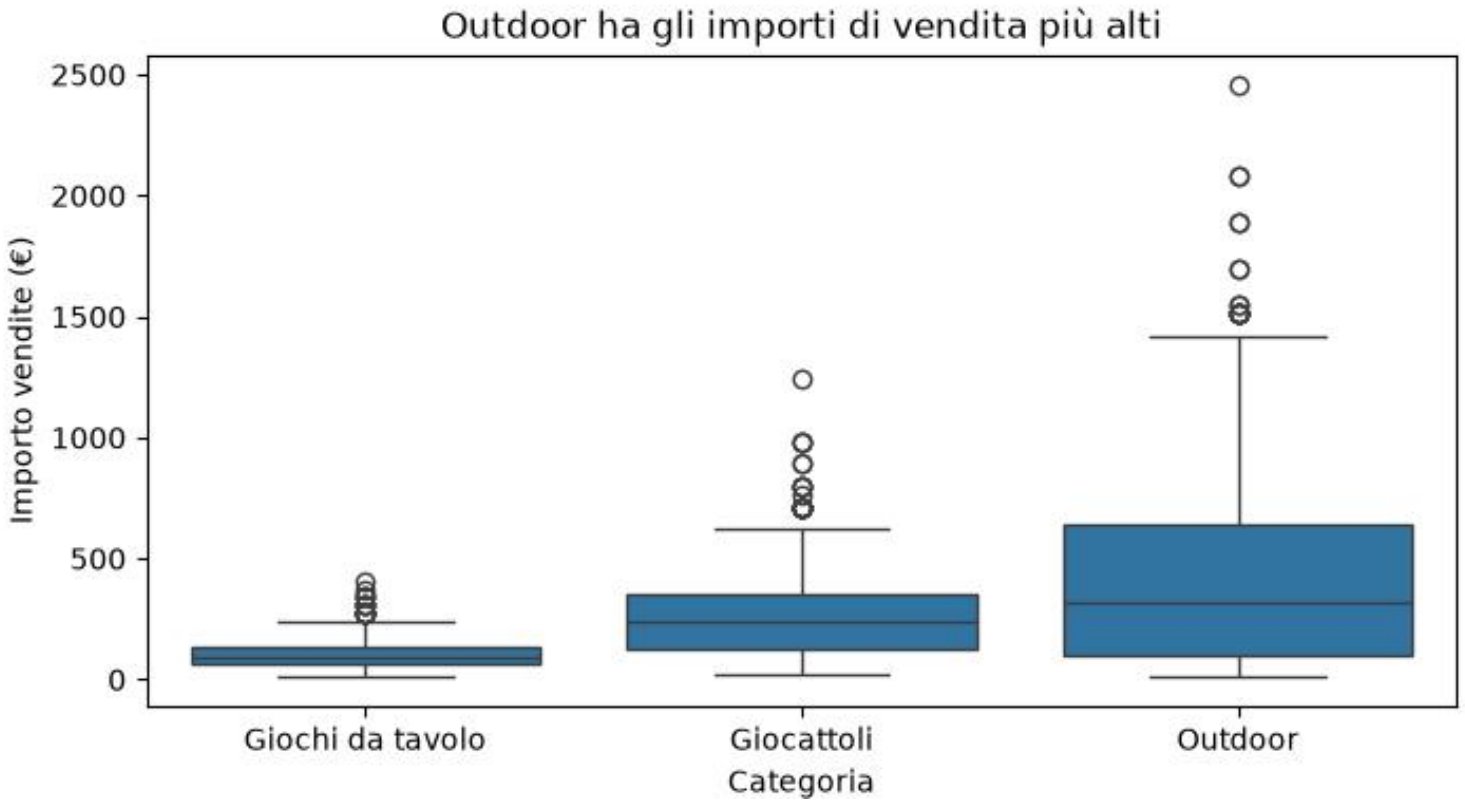
Il picco è stato nel maggio 2025 con 29.368 €; l'andamento è sempre molto oscillante da un mese all'altro e, col tempo, sembra mostrare una tendenza leggermente crescente.



Importi di vendita più alti

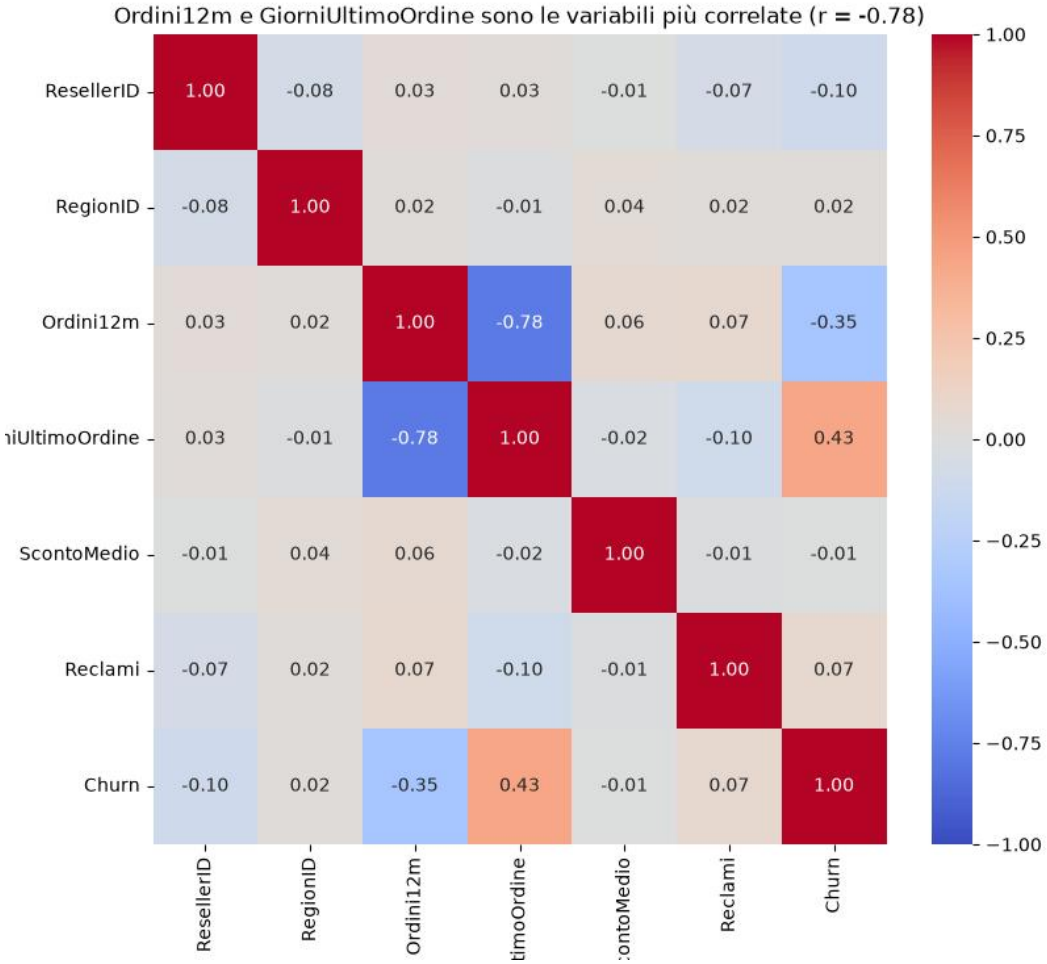
Gli importi delle vendite sono simili tra le tre categorie?

No, gli importi non sono simili, in quanto Outdoor ha gli importi mediani più alti ma anche più dispersi; dopo c'è Giocattoli, infine Giochi da tavolo che ha importi bassi e tutti molto concentrati.



Heatmap dei rivenditori più correalti

La coppia più correlata è Ordini12m e GiorniUltimoOrdine con (r = -0.78, negativa): in altre parole, vuol dire che chi ordina di più ha ordinato più recentemente nel tempo.



Task 5, *Correlazione e regressione lineare*

Realizzati i grafici, il prossimo Task riguarda l'analisi statistica: stimare il fatturato mensile con un modello lineare e usarlo per una previsione dei mesi successivi.

- Calcolare la correlazione tra fatturato mensile, quantità mensile e numero di rivenditori attivi nel mese, indicando la coppia più correlata.
- Realizzare una *Regressione Lineare* o semplice su (fatturato su indice del mese) e una *Regressione multipla* (indice del mese, quantità, rivenditori attivi).
- Calcolare *L'intercetta*, *i coefficenti* e R^2 per i due modelli così da prevedere, con il modello semplice, il fatturato dei tre mesi successivi all'ultimo disponibile.

Partiamo dalla regressione semplice

Questo modello cerca di capire se esiste una relazione diretta tra il tempo e l'andamento del fatturato. Immaginate di tracciare una linea retta che attraversa i tuoi dati storici per catturare il trend.

- L'obiettivo: Prevedere il fatturato futuro basandosi solo sul "numero del mese".
- La formula teorica è *Fatturato uguale intercetta più Coefficiente per Indice Mese*.

Qualche definizione

- *Intercetta* (Valore di partenza): Rappresenta il valore teorico del fatturato all'inizio del conteggio (quando l'indice del mese è pari a 0).
- *Coefficiente* (La pendenza): Ti dice di quanto aumenta (o diminuisce) mediamente il fatturato ogni mese che passa. Se il coefficiente è +500, significa che ogni mese il fatturato cresce di circa 500 unità.
- L'R2 misura quanta parte della variazione del fatturato il modello riesce a spiegare. La misura va da 0 a 1, se è veicina a 1 il modello spiega quasi tutto, se viceversa è vicina allo 0 spiega poco
- Una correlazione o R2 alti dicono soltanto che ci sono due numeri, grandezze, che si muovono insieme, ma non dicono nulla se ci sono rapporti causali dell'uno sull'altra: possono essere influenzate da una terza variabile, avere un legame casuale, anche.

Regressione multipla

Il fatturato reale spesso non dipende solo dal tempo, ma anche da fattori commerciali. Questo modello aggiunge più "indizi" contemporaneamente per fare una previsione più ricca e realistica.

- L'obiettivo: Spiegare il fatturato usando tre variabili insieme.
- Formula teorica: Fatturato = Intercetta + C_1(Indice Mese) + C_2 (Quantità) + C_3 (Rivenditori Attivi)

I risultati del Task 5

Anno	Mese	Fatturato	Quantità	Rivenditori	Indice
2023	1	11499.8	214	51	1
2023	2	9414.9	171	43	2
2023	3	14917.2	223	48	3
2023	4	11077.9	220	51	4
2023	5	10806.7	188	49	5

Table: Dati storici ridotti

	Fatturato	Quantità	Rivenditori
Fatturato	1.000	0.868	0.556
Quantità	0.868	1.000	0.673
Rivenditori	0.556	0.673	1.000

Table: Matrice di correlazione

I risultati del Task 5, confronto tra i modelli di regressione

Modello	Intercetta	C. Mese	C. Qta	C. Rivend	R2 Train	R2 Test
Semplice	10602.53	303.58	—	—	0.477	-1.879
Multipla	6323.67	-78.11	85.31	-229.36	0.836	-0.415

Table: Confronto metriche dei modelli di regressione

Infine, sulla base del modello semplice, abbiamo stimato per i tre mesi successivi della serie con valori 37, 38, 39:

- 21.834,88 € (gennaio 2026), - 22.138,46 € (febbraio 2026), - 2.442,04 € (marzo 2026).

Il coefficiente di IndiceMese è 303,58: in altre parole, la media del fatturato mensile aumenta di circa 303,58 € per ogni mese.

Task 6a, *Classificazione del churn dei rivenditori*

L'ultima parte della nostra analisi riguarda il passaggio alla *Regressione logistica* per prevedere quali rivenditori smetteranno di ordinare. Lavoreremo, quindi, nel campo del *Machine Learning*: per introdurre le richieste che ci sono state poste faremo un piccolo gioco mentale:

Immaginate che l'obiettivo sia addestrare una sorta di *investigatore virtuale* (che noi chiameremo *modello*) per capire in anticipo quali clienti stanno per smettere di ordinare da ToysGroup (che noi chiameremo *Churn*).

Le richieste del Task 6a

Al fine di rendere realmente comprensibile il lavoro svolto tramite il *Machine Learning*, riassumiamo in modo informale le 4 richieste che ci sono state fatte.

- Prepareremo i dati ed evitare di *barare*.
- Faremo fare delle previsioni all'investigatore , quindi al modello, usando delle *soglie*.
- Valuteremo l'efficacia del modello tramite tre metriche e la baseline che ci viene chiesta.
- Faremo una diagnosi: studieremo come si comporta il modello.

Il *Machine Learning*

Il *Machine learning* è un sottoinsieme dell'intelligenza artificiale che consente ai computer di apprendere dai dati e migliorare le proprie prestazioni nel tempo.

Invece di seguire un codice rigido basato su regole fisse (come, ad esempio, se accade A, allora fai B), i sistemi di *Machine learning* analizzano grandi quantità di informazioni, identificano schemi ricorrenti e creano modelli statistici autonomi per prendere decisioni o fare previsioni su dati totalmente nuovi.

Il termine è stato coniato per la prima volta nel 1959 dall'informatico americano Arthur Samuel.

Apprendimento supervisionato e non supervisionato

Come funzionano, però, questi modelli di apprendimento che andremo a impiegare?

Nell'*apprendimento supervisionato* il modello si addestra su dati il cui il target è noto: l'obiettivo è imparare e prevederlo con dati nuovi. Nel nostro scenario dobbiamo prevedere se un rivenditore smetterà di ordinare a partire da caratteristiche note di Churn e dei rivenditori.

Nell'*apprendimento non supervisionato*, invece, i dati non hanno un target noto: lo scopo dell'algoritmo è individuare strutture nascoste o clustering per riassumere l'informazione. Nel nostro scenario lo scopo è raggruppare i rivenditori per gruppo simili di ordini, sconti e reclami, con Churn non noto.

I Risultati

- Soglia Standard (0.5) – Approccio Prudente:
  Adottando un criterio rigido, il modello genera pochissimi falsi allarmi (4 casi), garantendo un'elevata precisione sui negativi. Tuttavia, si dimostra troppo conservativo, mancando l'identificazione di ben 10 casi positivi reali su 14 totali.

- Soglia Abbassata (0.3) – Approccio Sensibile:
  Abbassando l'asticella al 30%, il modello diventa più reattivo: i mancati allarmi scendono drasticamente da 10 a 4, intercettando la maggior parte dei positivi reali. Il prezzo da pagare è un incremento dei falsi allarmi, che salgono a 10.

Conclusione: La scelta della soglia non è un problema matematico, ma strategico. Si opta per la soglia 0.3 quando il costo di perdersi un evento reale è superiore al fastidio di gestire un falso allarme.

Ancora sui risultati

Il confronto delle metriche di classificazione evidenzia un comportamento analitico peculiare del modello, strettamente legato alla scelta della soglia decisionale rispetto alla baseline di riferimento.

- Il Vincolo dell'Accuracy e la Baseline (0.767): L'accuratezza complessiva del modello rimane identica (76.7%) sia con soglia 0.5 sia con soglia 0.3, e coincide esattamente con la baseline del modello ingenuo (che prevede sempre l'assenza di Churn). Questo indica che lo sbilanciamento delle classi domina la metrica globale, rendendo l'accuratezza un indicatore insufficiente per valutare il reale miglioramento del modello.

Ancora sui risultati

- La Stabilità della Precision (0.500):In entrambi gli scenari, la precisione rimane fissa al 50%. Ciò significa che indipendentemente dall'ampiezza dei casi intercettati, la metà delle segnalazioni di Churn effettuate dal modello risulterà comunque un falso allarme.
- Il Turnaround del Recall (0.286 vs 0.714): La vera differenza si focalizza sul Recall. Con la soglia standard a 0.5, il modello intercetta appena il 28.6% dei clienti in uscita. Abbassando la soglia a 0.3, la capacità di catturare il Churn reale crolla positivamente verso il 71.4%, triplicando l'efficacia del recupero aziendale a parità di accuratezza generale.

Task 6b, *Segmentazione dei rivenditori con K-Means*

L'ultima parte del lavoro con il nostro *Investigatore* o *modello* riguarda la suddivisione dei rivenditori in segmenti omogenei per comportamento d'acquisto e confrontare i segmenti con il *Churn*, introdotto prima.

Questa volta l'obiettivo non è fare previsioni, come in precedenza, ma mettere ordine e fare degli *identikit*. Immaginate, quindi, di avere una folla disordinata di rivenditori e di volerli dividere in *gruppi di amici* (che noi chiameremo *i cluster*) che si comportano nello stesso modo quando comprano, per poi vedere quali di questi gruppi sono più a rischio di abbandonarti (quindi il *Churn*).

Le richieste del Task 6b

Similmente a prima, per concludere, anche qua spiegheremo informalmente le ultime richieste poste per l'analisi dello scenario *ToysGroup*:

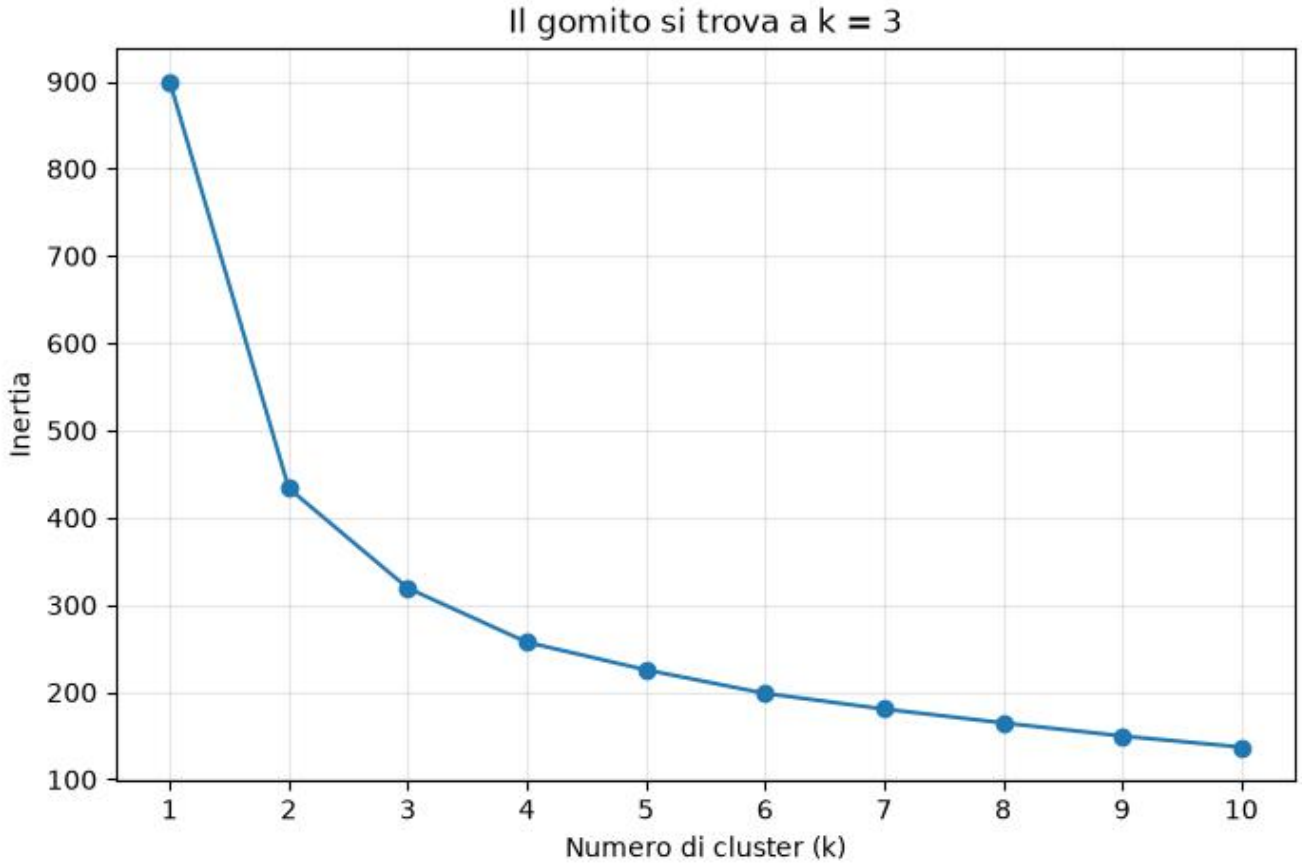
- Faremo calcoli sui i dati, saranno livellati per fare la *prova dei gruppi*.
- Sceglieremo il numero perfetto di gruppi tramite il *metodo del gomito*
- Faremo l'identikit dei gruppi per scoprire chi rischia di abbandonare.

Alcune definizioni

- Il *centroide* è il punto medio di un cluster che viene calcolato sulla media delle variabili dei rivenditori che fanno parte di quel cluster.
- L'*inerzia* è, matematicamente, la somma delle distanza al quadrato tra ogni rivenditori e il punto medio di un cluster (il centroide), in altre parole, più l'inerzia è bassa più i gruppi risultano essere compatti.
- A differenza dei precedenti task, K-Means non richiede una colonna y perché non c'è nessun target noto: cerca soltanto rivenditori simili tramite le variabili di comportamento. Di conseguenza, la divisione train/test è superflua non essendoci previsioni da verificare su dati nuovi. Il Churn, quindi, lo usiamo solo dopo per confrontare i segmenti risultanti.

Grafico del gomito

Il grafico del gomito fa vedere come l'inerzia scende rapidamente fino a k = 3, quindi da 900 a 300 circa, e poi va più lentamente da k = 4 in poi.



Le ultime analisi sui gruppi

L'algoritmo di clustering ha identificato tre profili aziendali distinti, permettendo di mappare il comportamento dei clienti e di associare a ciascun gruppo una precisa etichetta strategica basata sulle metriche operative e sul tasso di abbandono.

- Il Segmento dei Dormienti (Cluster 1):
  Questo gruppo rappresenta la criticità maggiore per l'azienda, registrando un allarmante tasso di Churn del 39%. Pur contando sul maggior numero di rivenditori (137) e su una buona frequenza storica (12.42 ordini), i clienti mostrano una prolungata inattività con una media di ben 141.48 giorni dall'ultimo ordine e il fatturato più basso della serie (1288.47 €).

Infine, normali e dormienti

- Il Segmento dei Normali (Cluster 0):
  Identifica la spina dorsale della clientela attiva. Mostra un'inattività moderata (48.35 giorni dall'ultimo acquisto), un volume di circa 19.58 ordini e un fatturato di 1514.70 € distribuito su 95 rivenditori. Il tasso di Churn si attesta su livelli fisiologici e stabili pari all'11%.
- Il Segmento dei Top(Cluster 2):
  Rappresenta lo zoccolo duro ad alto valore. Pur registrando il minor numero di rivenditori (68), questo cluster genera il fatturato più elevato in assoluto (3524.38 €) con il maggior numero di ordini medi (21.66) e la massima reattività d'acquisto (solo 45.90 giorni dall'ultimo ordine). Il rischio di Churn è minimo, pari ad appena il 10%.

Osservazioni finali

Diamo, infine, uno sguardo su quanto abbiamo fatto.

Il lavoro è stato svolto utilizzando il linguaggio di programmazione Python sul data base dell'Azienda ToysGroup con relativo scenario, che abbiamo analizzato in questa presentazione

È molto probabile che nella risoluzione della task che ci sono state assegnate non abbiamo sempre usato il metodo di risoluzione migliore: anzi, alcune volte è stata tralasciata sicuramente la strada migliore per una probabilmente più complessa e meno intuitiva.

Sulla presentazione

All'interno della nostra presentazione, invece, abbiamo completamente abbandonato la spiegazione della sintassi di Python: , abbiamo scelto il metodo di spiegare *cosa* abbiamo utilizzato per calcolare *cosa*, spiegando dunque ii risultati ottenuto. Molte cose, quindi, le abbiamo tralasciate consapevoli, però, della strada che abbiamo scelto di intraprendere.

Speriamo quindi, in conclusione, nella buona riuscita di quanto è stato presentato.

Grazie!